

DA5300: Data Structures For Data Science

Laboratory 2 – Python Data Structures

Objective

The objective of this laboratory is to understand how Python stores objects in memory. In this laboratory, you will learn the following concepts.

- Low-level Arrays
- Referential Arrays
- Mutable and Immutable Objects
- Assignment, Shallow Copy and Deep Copy
- Dynamic Arrays using `ctypes.py_object()`

All exercises are based on the theme of maintaining a cricket team database.

Exercise 1 – Low-Level Arrays

Suppose a cricket team's Playing XI is represented only by the jersey numbers of its players.

The jersey numbers are

18 45 63 1 33 77 8 99 90 93 19

Since jersey numbers are integers, they can be stored using the Python `array` module.

<pre>from array import array jersey = array('i', [18,45,63,1,33, 77,8,99,90,93,19])</pre>

Answer the following questions.

Q1.1 Print all jersey numbers.

Q1.2 Print the jersey number of

- the first player,
- the wicket keeper,
- the last player.

Q1.3 Replace the jersey number of one injured player.

Q1.4 Count the total number of players.

Q1.5 Compute the average jersey number.

Q1.6 Can player names be stored in the above array?

Q1.7 Can objects of class `Player` be stored in the above array?

Exercise 2 – Referential Arrays

Now each player has

- Name
- Jersey Number

Implement the following class.

```
class Player:

    def __init__(self, name, jersey):

        self.name = name
        self.jersey = jersey
```

Create a list containing Player objects.

```
players = [

    Player("Virat",18),
    Player("Rohit",45),
    Player("Gill",77),
    Player("Bumrah",93)

]
```

Answer the following questions.

Q2.1 Print

```
id(players)
```

Q2.2 Print

```
id(players[0])

id(players[1])

id(players[2])

id(players[3])
```

Q2.3 Print

```
id(players[0].name)

id(players[0].jersey)
```

Q2.4 Modify the name of the first player and verify that the modification is visible through the list.

Exercise 3 – Mutable and Immutable Objects

Part A – Immutable Objects

Consider the following code.

```
captain = "Virat"

vice_captain = captain
```

Answer the following questions.

Q3.1 Print

```
id(captain)

id(vice_captain)
```

Q3.2 Execute

```
captain = "Gill"
```

Again print

```
id(captain)

id(vice_captain)
```

Q3.3 Explain why the value of `vice_captain` did not change.

Part B – Mutable Objects

Consider the following code.

```
test_squad = [

    "Virat",
    "Gill",
    "Pant"

]

backup = test_squad
```

Answer the following questions.

Q3.4 Append

```
"Bumrah"
```

to

```
test_squad
```

Q3.5 Print both variables.

Q3.6 Print

```
id(test_squad)

id(backup)
```

Q3.7 Explain why both variables changed after modifying `test_squad`.

Exercise 4 – Assignment, Shallow Copy and Deep Copy

Suppose the Indian cricket team is represented using a nested list.

```
india = [  
    ["Virat", "Gill", "Pant"],  
    ["Bumrah", "Siraj", "Shami"]  
]
```

The first list contains the batters and the second list contains the bowlers.

Part A – Assignment

Execute the following statement.

```
team2 = india
```

Answer the following questions.

Q4.1 Append

```
"Jaiswal"
```

to the batting list.

Q4.2 Print

```
india  
  
team2
```

Q4.3 Print

```
id(india)  
  
id(team2)
```

Q4.4 Draw the memory diagram showing how `india` and `team2` are stored in memory.

Q4.5 Explain why both variables changed.

Part B – Shallow Copy

Execute

```
import copy  
  
team3 = copy.copy(india)
```

Answer the following questions.

Q4.6 Append

```
"Kuldeep"
```

to the bowling list.

Q4.7 Print

```
india  
team3
```

Q4.8 Print

```
id(india)  
id(team3)
```

Q4.9 Print

```
id(india[0])  
id(team3[0])  
id(india[1])  
id(team3[1])
```

Q4.10 Draw the corresponding memory diagram.

Q4.11 Explain why the outer lists are different while the inner lists are shared.

Part C – Deep Copy

Execute

```
team4 = copy.deepcopy(india)
```

Answer the following questions.

Q4.12 Append

```
"Rahul"
```

to the batting list.

Q4.13 Print

```
india  
team4
```

Q4.14 Print

```
id(india)  
id(team4)  
id(india[0])  
id(team4[0])  
id(india[1])  
id(team4[1])
```

Q4.15 Draw the corresponding memory diagram.

Q4.16 Explain why modifying one list does not affect the other.

Exercise 5 – Implementing a Dynamic Array

In this exercise you will implement your own version of Python's `list`.

The array stores objects of class `Player`.

Memory must be allocated using

```
(ctypes.py_object * capacity)()
```

Implement a class named

```
class SquadArray
```

and complete the following tasks.

Q5.1 Implement the constructor.

Initialize

```
size = 0  
capacity = 1
```

Q5.2 Implement

```
__len__()
```

to return the current number of players.

Q5.3 Implement

```
append(player)
```

to insert a new player at the end of the array.

Q5.4 Whenever the array becomes full, allocate a new array having twice the current capacity and copy all elements into the new array.

Q5.5 Implement

```
resize(new_capacity)
```

using

```
(ctypes.py_object * new_capacity)()
```

Q5.6 Implement

```
__getitem__(index)
```

to access a player using indexing.

Q5.7 Implement

```
remove(player_name)
```

to remove a player from the array.

Q5.8 Implement

```
display()
```

to print all players currently present in the array.

Q5.9 Insert twenty players into the dynamic array.

After every insertion print

```
Current Size
```

```
Current Capacity
```

Construct a table of the form

Number of Players	Capacity
1	1
2	2
3	4
4	4
5	8
$\vdots$	$\vdots$
20	?

Q5.10 Based on your observations, answer the following.

- (a) What is the worst-case time complexity of `append()`?
- (b) What happens if capacity is increased by size one after every insertion instead of doubling?